



L'assembleur nasm 64 bits

Registres

Boucles

Sommaire

Généralités

Registres

Boucles

Accès à la mémoire

Terminaison du programme

Pourquoi apprendre un assembleur ?

```
.section .text
org 0x100
    mov ah, 0x9
    mov dx, hello
    int 0x21
    mov ax, 0x4c00
    int 0x21
section .data
hello: db 'Hello, world!', 13, 10, '$'
```

Un programme en **nasm** pour Windows

Comprendre comment fonctionne un processeur
Écrire du code efficace
S'amuser

Docs, tutoriels, manuels, livres

<https://www.felixcloutier.com/x86/index.html>

http://esauvage.developpez.com/tutoriels/asm/assembleur-intel-avec-nasm/?page=page_1

http://www.pravaraengg.org.in/Download/MA/assembly_tutorial.pdf

<https://cs.lmu.edu/~ray/notes/nasmtutorial/>

<http://www.nasm.us/doc/>

<http://www.nasm.us/doc/nasmdocb.html>

<http://pacman128.github.io/static/pcasm-book.pdf>

<http://asmtutor.com/>

<http://www.davidsalomon.name/assem.advertis/asl.pdf>

Documentation des instructions



Étienne Sauvage. 2011. *Assembleur Intel avec NASM*.

Assembly Language Tutorial, Tutorialspoint.

Ray Toal. *NASM Tutorial*, Loyola Marymount University.

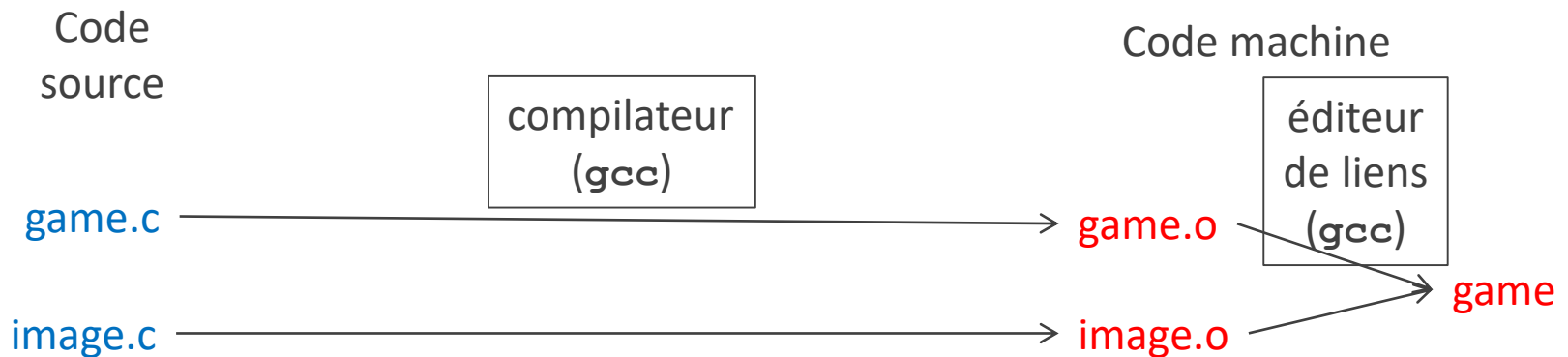
The Netwide Assembler: NASM, documentation de NASM.

Paul Carter. 2006. *PC Assembly Language* (Langage d'assemblage pour PC).

Daniel Givney. 2013. *Learn Assembly Language*.

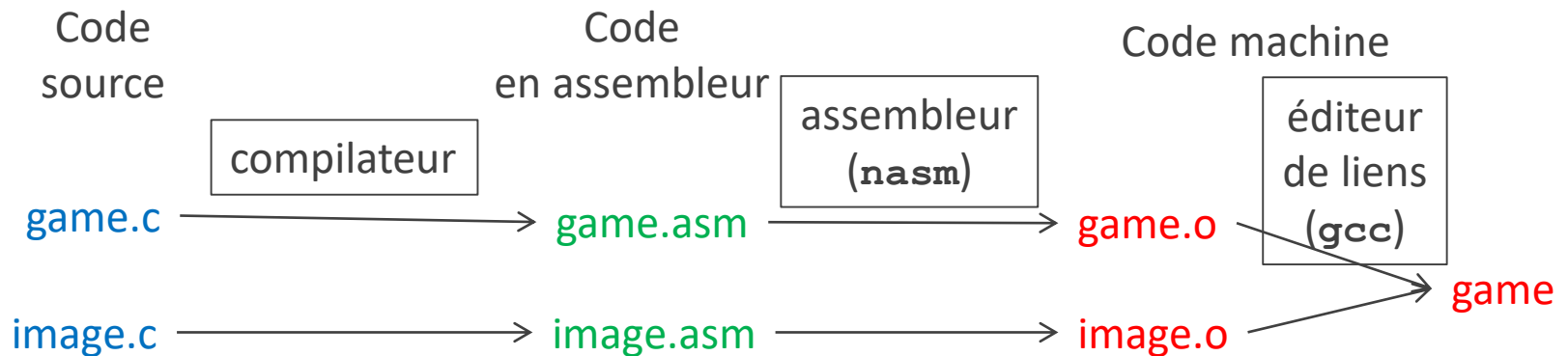
David Salomon. 1993. *Assemblers and Loaders*, Ellis Horwood.

Chaine de traitement



```
gcc -c game.c -Wall
gcc -c image.c -Wall
gcc -o game game.o image.o -Wall
```

Chaine de traitement



Extension : souvent `.asm` ou `.s`

Chaine de traitement

Assembleur :
programme qui
traduit en binaire

Code
en assembleur

Code machine

assembleur

éditeur
de liens
(gcc)

game.asm

game.o

image.asm

image.o

game

Assembleur ou langage
d'assemblage (*assembly
language*)

Chaque assembleur (programme) définit un
assembleur (langage) qu'il peut traduire en
binaire
Les premiers assembleurs faisaient aussi l'édition
de liens

```
; hello.asm
section .text
org 0x100
    mov ah, 0x9
    mov dx, hello
    int 0x21
    mov ax, 0x4c00
    int 0x21
section .data
hello: db 'Hello, world!', 13, 10, '$'
```

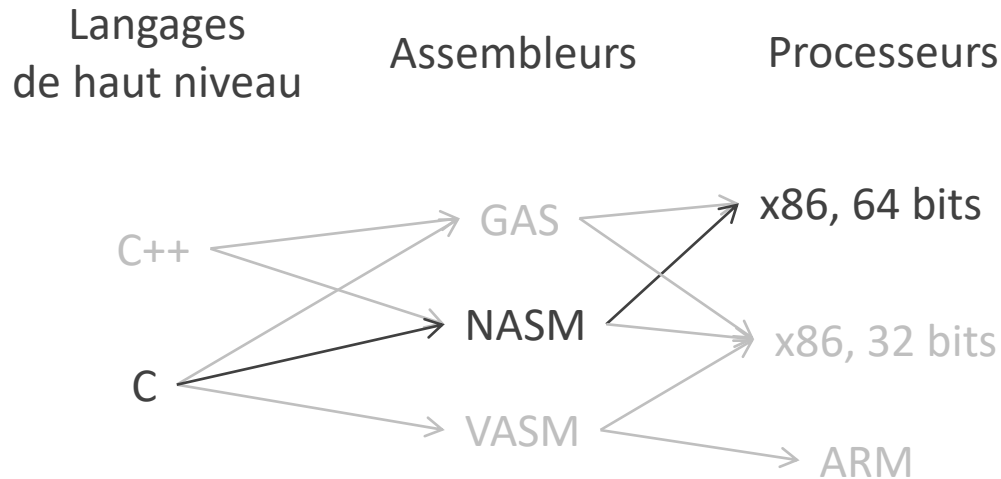
Un programme en `nasm` pour Windows

nasm

Netwide Assembler

Un assembleur open-source pour processeurs x86

Autres assembleurs et processeurs



Un langage de haut niveau peut être compilé en plusieurs assembleurs

Un assembleur peut être compilé pour plusieurs processeurs

Lancer l'assembleur

Formats de fichiers objet

```
nasm -f elf64 my_prog.asm
```

```
global _start
section .text
_start:
    mov eax, 1 ; write
    mov edi, 1 ; stdout
    mov rsi, msg
    mov edx, msg.len
    syscall ; write
    mov eax, 60 ; exit
    mov edi, 0 ; all correct
    syscall ; exit(0)
section .data
msg: db "Hello, world!", 10
.len: equ $ - msg
```

Dépendent du système d'exploitation

elf64	Linux 64 bits
elf32	Linux, simuler un processeur 32 bits
win64	Windows 64 bits
win32	Windows 32 bits
bin	MS-DOS
...	

Les règles du langage `nasm` dépendent du format visé

Exemple : la directive `org` (diapo 8) n'est pas compatible avec le format **elf64**

Un programme en `nasm` pour Linux

Lancer l'édition de liens

```
gcc -o my_prog my_prog.o utils.o -nostartfiles -no-pie
```

```
global _start
section .text
_start:
    mov eax, 4 ; write
    mov ebx, 1 ; stdout
    mov ecx, msg
    mov edx, msg.len
    (...)
```

Avec l'option `-nostartfiles`, l'exécution commence à l'étiquette `_start`, sinon il y a un `crt0.o` qui appelle `main()`

Sans l'option `-no-pie` (*position-independent executable*), le programme peut être chargé à n'importe quelle adresse (fonctionnalité peut ne pas être opérationnelle sur les machines de l'Université)

Commentaires

```
global _start
section .text
_start:
    mov eax, 4 ; write
    mov ebx, 1 ; stdout
    mov ecx, msg
    mov edx, msg.len
    int 0x80 ; write
    mov eax, 1 ; exit
    mov ebx, 0
    int 0x80 ; exit(0)
section .data
msg: db "Hello, world!", 10
.len: equ $ - msg
```

Depuis ";" jusqu'à la fin de la ligne
Ce n'est pas un smiley

Sommaire

Généralités

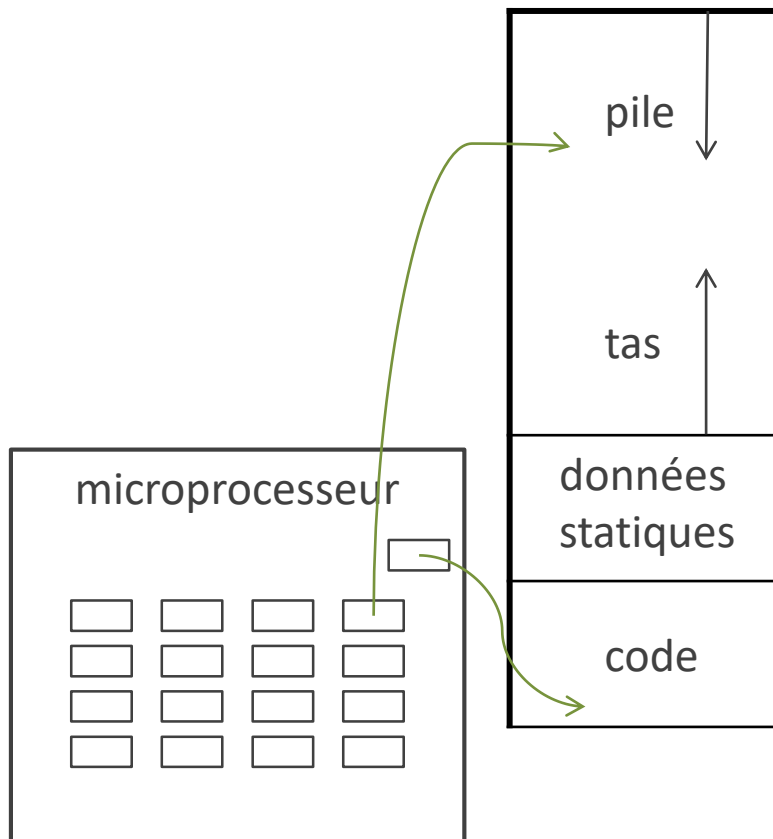
Registres

Boucles

Accès à la mémoire

Terminaison du programme

Registres



```
global _start
section .text
_start:
    mov eax, 4 ; write
    mov ebx, 1 ; stdout
    mov ecx, msg
    mov edx, msg.len
```

Équivalent des variables en C
mais font partie du processeur et non de la
mémoire centrale

Le compteur ordinal est un registre géré
uniquement par le moteur d'exécution

Les principaux registres

8 octets	4 octets	2 octets	1 octet	1 octet
rbx	ebx	bx	bh	bl
r12	r12d	r12w		r12b
r13	r13d	r13w		r13b
r14	r14d	r14w		r14b
r15	r15d	r15w		r15b

Compatibilité descendante avec les programmes en versions 32 bits, 16 bits, 8 bits

r : register

e : extended

h : high

l : low

d : double word (= 4 octets)

w : word (= 2 octets)

b : byte

Copier dans un registre : l'instruction **mov**

Une constante en hexadécimal
doit commencer par un chiffre

aah identificateur
0aah constante numérique

Constantes ("données immédiates")

mov rbx, 60	copier 60 dans rbx
60	décimal
3ch	hexadécimal
0x3c	hexadécimal
0h3c	hexadécimal
00111100b	binaire
0b0011_1100	binaire

Registres

mov rbx, r12	copier r12 dans rbx
---------------------	-----------------------------------

Instructions arithmétiques

```
mov rbx, 60
add rbx, 20
mov r12, 40
add rbx, r12
dec rbx
```

```
add x, y      ; x := x+y
sub x, y      ; x := x-y
```

```
inc x         ; x := x+1
dec x         ; x := x-1
```



By PiccoloNamek (English wikipedia) [GFDL (<http://www.gnu.org/copyleft/fdl.html>) or CC-BY-SA-3.0 (<http://creativecommons.org/licenses/by-sa/3.0/>)], via Wikimedia Commons

Registres volatils

Il y a d'autres registres que ceux de la diapo 15, par exemple `rax`, mais leur valeur peut être écrasée quand le code `nasm` appelle une fonction en C : **registres volatils**

Dans les deux premiers TP d'assembleur, on utilisera des entrées-sorties qui provoquent un appel de `printf()`

Pour l'instant, utilisez seulement des **registres non volatils**

```
show_registers:
    push rbp
    mov rbp, rsp
    mov r8, r14
    mov rcx, r13
    mov rdx, r12
    mov rsi, rbx
    mov rdi, format_registers
    mov rax, 0
    call printf
    pop rbp
    ret
```

```
mov rbx, 60
mov r12, 40
mov r13, 20
mov r14, 10
call show_registers
```

Entrées-sorties

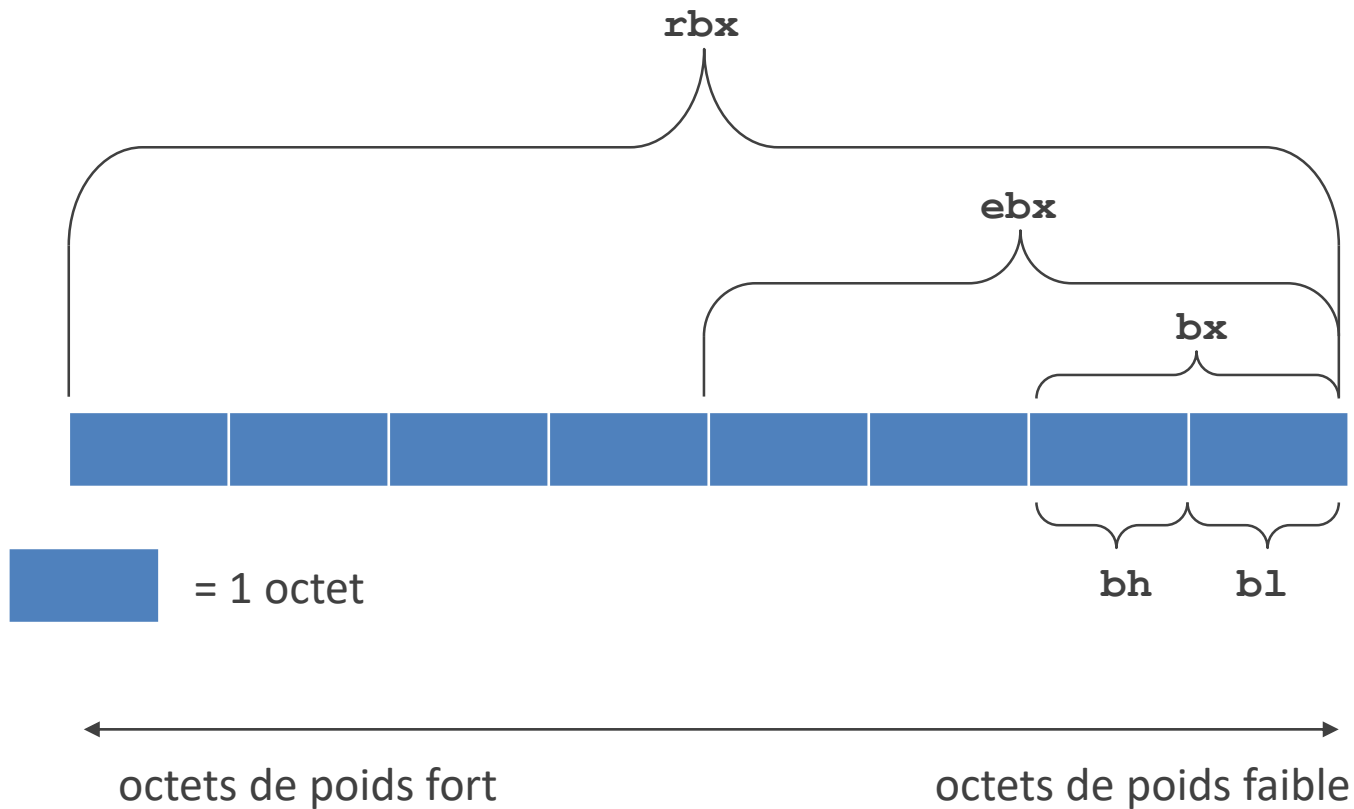
Entrées-sorties : lire ou écrire dans des fichiers,
dans l'entrée standard ou la sortie standard

Ce sont des appels système

Uniquement chaînes de caractères

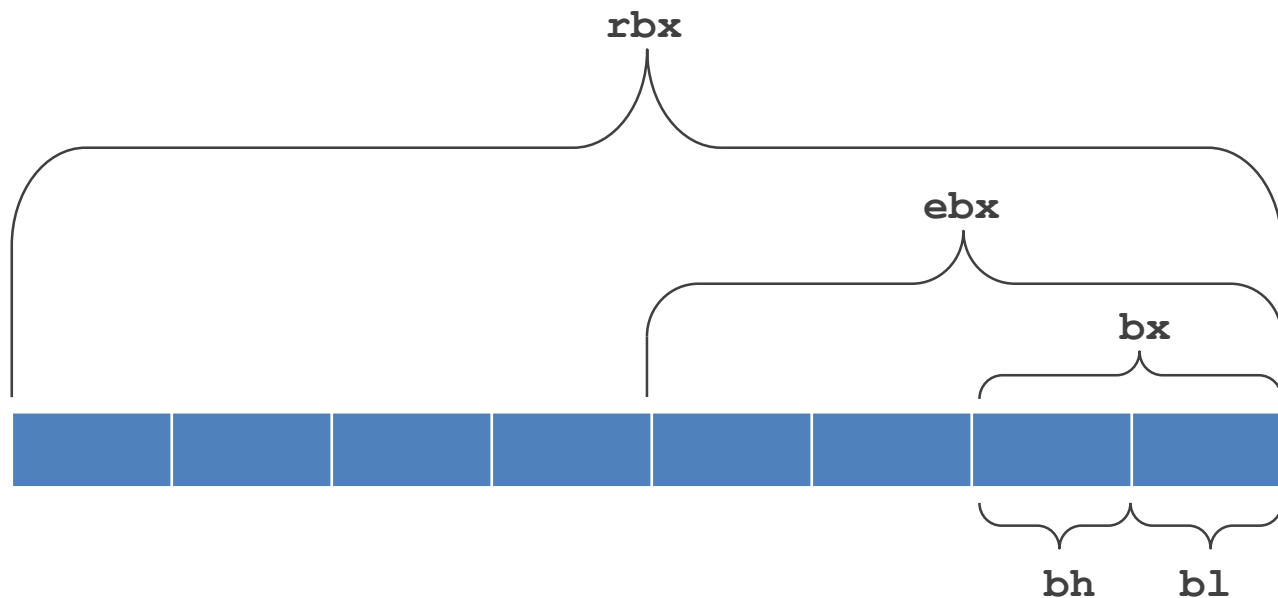
Pour les deux premiers TP d'assembleur, on fera
les entrées-sorties en appelant une fonction
`show_registers` qui elle-même appelle
`printf()`

Chevauchement (*overlapping*) entre registres



Exercice : étant donné la valeur d'un registre, déduire celle d'un sous-registre

Chevauchement (*overlapping*)

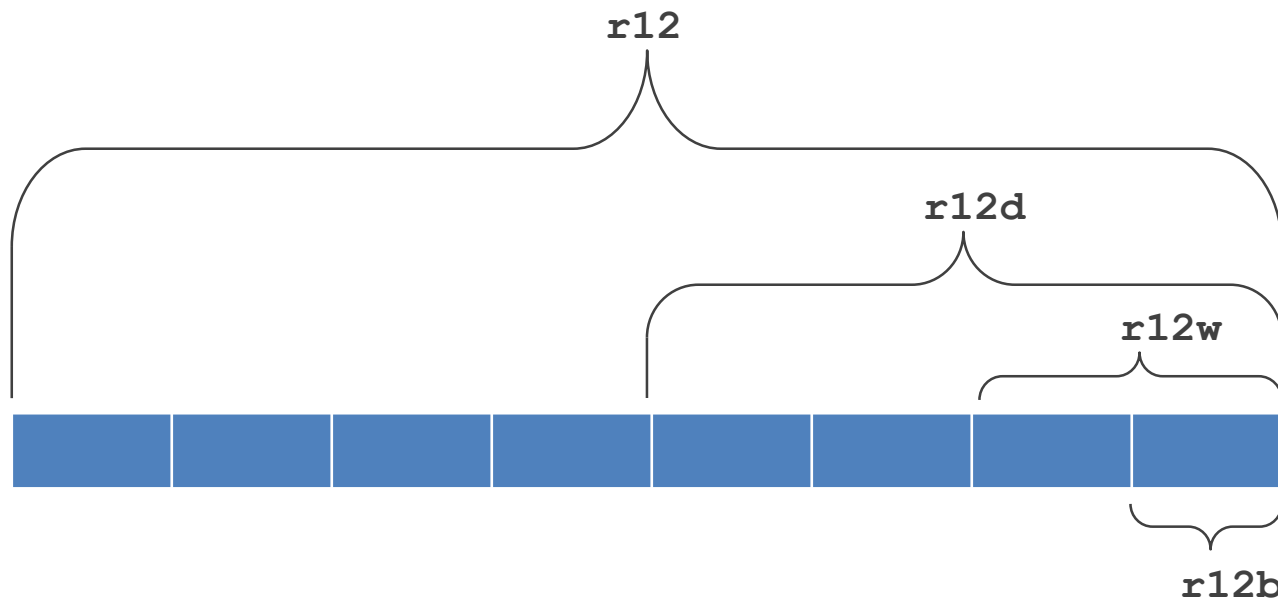


```
mov rbx, 60
mov ebx, 40
call show_registers
```

Si on modifie **ebx**, ça modifie **rbx**

Si on modifie **rbx**, ça peut modifier **ebx**

Chevauchement (*overlapping*)

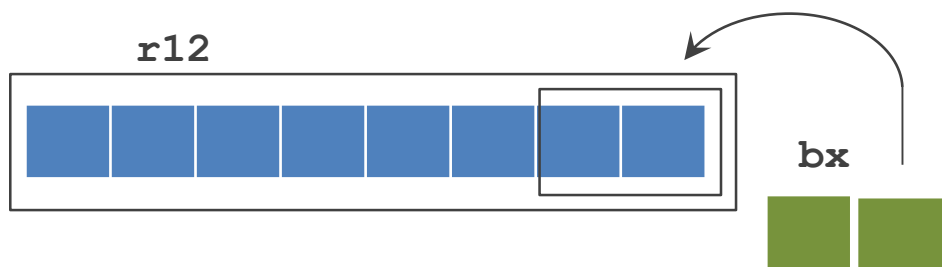


```
mov r12, 60
mov r12d, 40
call show_registers
```

Si on modifie **r12d**, ça modifie **r12**

Si on modifie **r12**, ça peut modifier **r12d**

Opérandes de tailles différentes



~~mov r12, bx~~

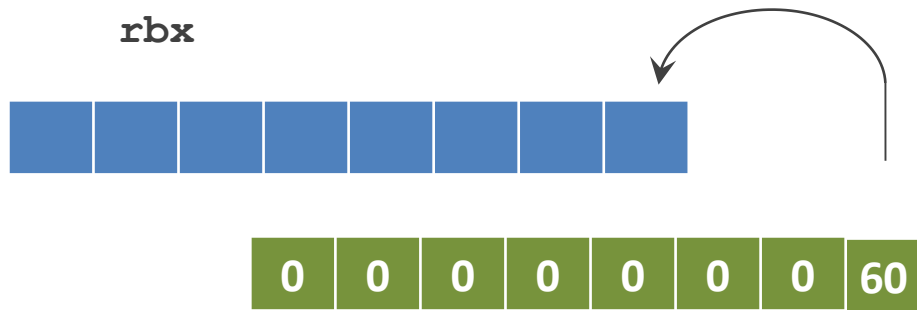
```
mov r12, rbx
mov r12d, ebx
mov r12w, bx
mov r12b, bh
mov r12b, bl
add rbx, r12
```

L'instruction `mov` va-t-elle écraser **tout** l'**opérande destination**, ou seulement les octets de poids faible ?

`nasm` a des règles différentes suivant les cas
Exemple : `mov r12, bx` est illégale à cause des opérandes de tailles différentes

Conseil : faites seulement des instructions avec des opérandes **de même taille**

Taille des opérandes

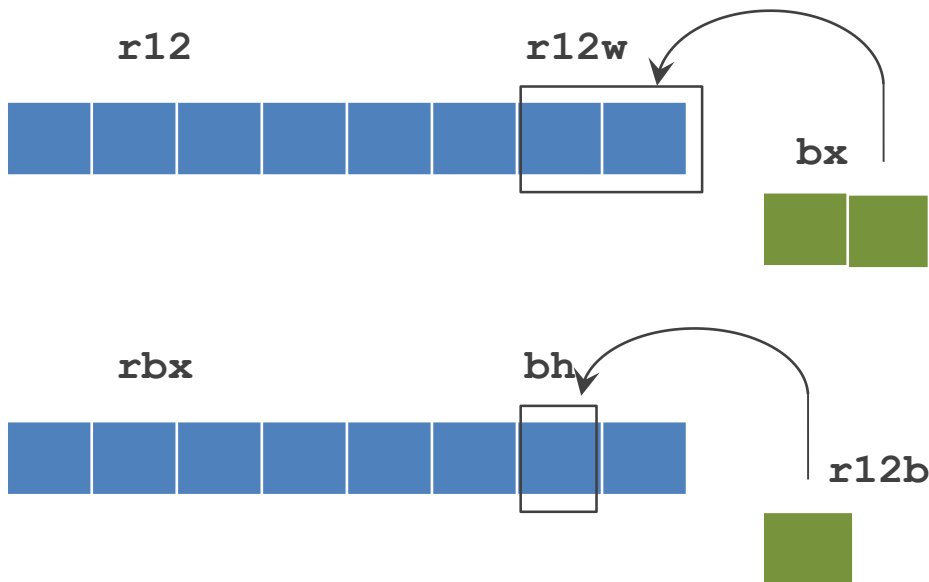


`mov rbx, 60`

La taille de la constante est fixée par la taille du registre

L'instruction `mov` écrase **tout l'opérande destination**, même si la constante tient sur une taille mémoire plus petite

Opérations sur registres de 1 ou 2 octets



```
mov r12w, bx ; écrase 2 octets
mov bh, r12b ; écrase 1 octet
```

Exercice : comment savoir quels octets de `r12` contiennent autre chose que des 0 ?

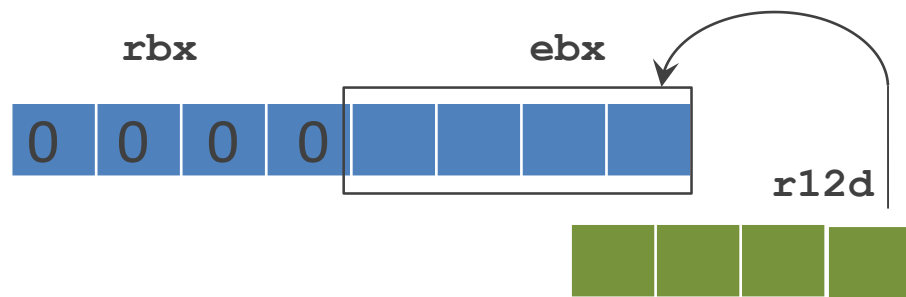
Convertir de l'hexadécimal en binaire

0x11 0xba 0xcf3b 0x4e43 0x1234 0xff 0xf34e

Convertir du décimal en hexadécimal

128 256 512 270

Opérations sur registres de 4 octets



Si le premier opérande est un registre sur **4 octets**,
`mov`, `add`, `sub`, `inc`, `dec` mettent à zéro les
4 octets de poids fort

Compatibilité descendante avec le mode 32 bits

```
mov ebx, r12d ; écrase tout rbx
mov ebx, 60   ; écrase tout rbx
```

Exercice : mettre dans `r12` le reste de la division
de `rbx` par 2^{32}

Résumé : instructions de base

<code>mov</code>	move	<code>mov x, y</code>	copier <code>y</code> dans <code>x</code> (écrase <code>x</code>)
<code>add</code>	add	<code>add x, y</code>	<code>x := x+y</code>
<code>sub</code>	subtract		
<code>inc</code>	increment		
<code>dec</code>	decrement		
<code>call</code>	function call	<code>call x</code>	appeler une fonction <code>x</code>

Sommaire

Généralités

Registres

Boucles

Accès à la mémoire

Terminaison du programme

Branchement conditionnel

```

    global _start
_start:
    mov rbx, 1000
    mov r12, 7
my_first_loop_label:
    sub rbx, r12
    cmp rbx, 30
    jg my_first_loop_label
    mov r13, rbx

```

étiquette (*label*)

instruction de branchement

On passe au moins une fois dans cette boucle

La condition est à la fin de la boucle

Pas de { }

Pas de **begin end**

Indentation non significative

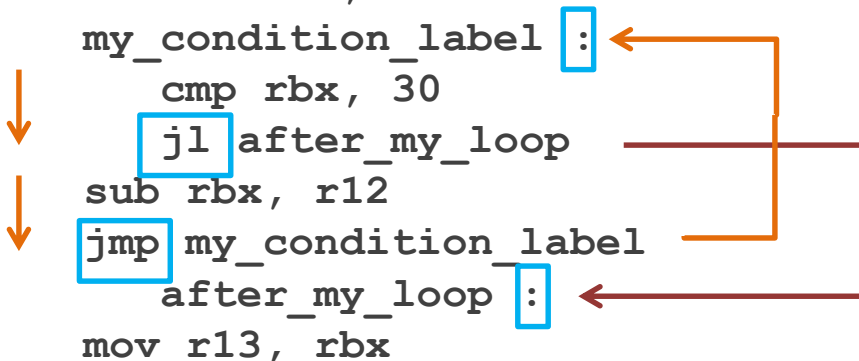
Branchement inconditionnel

```

    global _start
_start:
    mov rbx, 1000
    mov r12, 7
    mov r13, 0
    mov r14, 0
my_condition_label:
    cmp rbx, 30
    jl after_my_loop
    sub rbx, r12
    jmp my_condition_label
after_my_loop:
    mov r13, rbx

```

Pour une boucle dans laquelle on ne passe pas toujours, il faut 2 branchements et 2 étiquettes
avec la condition au début de la boucle



Branchement conditionnel après `cmp`

```
cmp rbx, 30
jg my_first_loop_label
```

L'instruction `jg` réalise le saut ou non suivant le résultat de l'instruction `cmp`

Branchement conditionnel :	<code>je</code>	<code>jne</code>	<code>jg</code>	<code>jge</code>	<code>jl</code>	<code>jle</code>
Après <code>cmp x, y</code> saut si :	$x=y$	$x \neq y$	$x > y$	$x \geq y$	$x < y$	$x \leq y$

Comment ça marche

L'instruction `cmp` touche plusieurs bits du registre `rflags`, entre autres

- le bit de signe (*sign flag*) : 0 si $x \geq y$
- le bit de zéro (*zero flag*) : 1 si $x=y$

L'instruction `jg` accède à `rflags`

Autres instructions qui touchent `rflags`

```
add rbx, 30
jg my_first_loop_label
```

Branchement conditionnel :	je	jne	jg	jge	jl	jle
Après add <code>x</code> , <code>y</code> saut si :	<code>x=0</code>	<code>x≠0</code>	<code>x>0</code>	<code>x≥0</code>	<code>x<0</code>	<code>x≤0</code>
Après sub <code>x</code> , <code>y</code> saut si :	<code>x=0</code>	<code>x≠0</code>	<code>x>0</code>	<code>x≥0</code>	<code>x<0</code>	<code>x≤0</code>
Après inc <code>x</code> saut si :	<code>x=0</code>	<code>x≠0</code>	<code>x>0</code>	<code>x≥0</code>	<code>x<0</code>	<code>x≤0</code>
Après dec <code>x</code> saut si :	<code>x=0</code>	<code>x≠0</code>	<code>x>0</code>	<code>x≥0</code>	<code>x<0</code>	<code>x≤0</code>

et beaucoup d'autres instructions

Instructions qui ne touchent pas `rflags`

```
cmp rbx, 30
add r12, r13
jg my_first_loop_label
```

```
cmp rbx, 30
mov r12, r13
jg my_first_loop_label
```

```
cmp rbx, 30
je my_label_for_equal
jg my_label_for_more
jl my_label_for_less
```

S'il y a une instruction entre `cmp` et `jg`, elle peut écraser les valeurs laissées par `cmp`

`mov` ne touche pas `rflags`

Les branchements conditionnels `je`, `jne`, `jg`, `jge`, `jl`, `jle` non plus

Entiers signés

```

mov rbx, -273
cmp 12, rbx
jg is_more
jl is_less
is_more:
    mov r12, 12
    jmp end
is_less:
    mov r12, rbx
end:
; r12 contains 12

```

Les branchements conditionnels **j~~e~~**, **j~~n~~e**, **j~~g~~**, **j~~g~~e**, **j~~l~~**, **j~~l~~e** considèrent les opérandes de **cmp**, **add**, **sub**, **inc**, **dec** comme des entiers signés codés en complément à 2

S'ils les considéraient comme des entiers non signés, le branchement sauterait à **is_less**

```
global show_registers
extern printf
    show_registers:
push rbp
mov rbp, rsp
(...)
mov rax, 0
call printf
pop rbp
ret
```

```
global _start
extern show_registers
_start:
    mov rbx, 1000
    mov r12, 7
    my_first_loop_label:
        sub rbx, r12
        cmp rbx, 30
        jg my_first_loop_label
    mov r13, rbx
    call show_registers
```

Étiquettes (*labels*)

Forme générale

<étiquette> ":"

Le ":" n'est pas obligatoire

Utilisation

Opérande d'un branchement
ou d'un `call`

Déclarations

`extern <étiquette>`

`global <étiquette>`

importer l'étiquette
exporter l'étiquette

Exercices

```
global _start
_start:
    mov rbx, 1000
    mov r12, 7
my_first_loop_label :
    sub rbx, r12
    cmp rbx, 30
    jg my_first_loop_label
mov r13, rbx
```

Exercice 1

Ajouter un compteur à la boucle ci-contre

Exercice 2

Écrire du code pour vérifier si les 4 octets de poids fort de `rbx` sont à zéro sans écraser `rbx`

Sommaire

Généralités

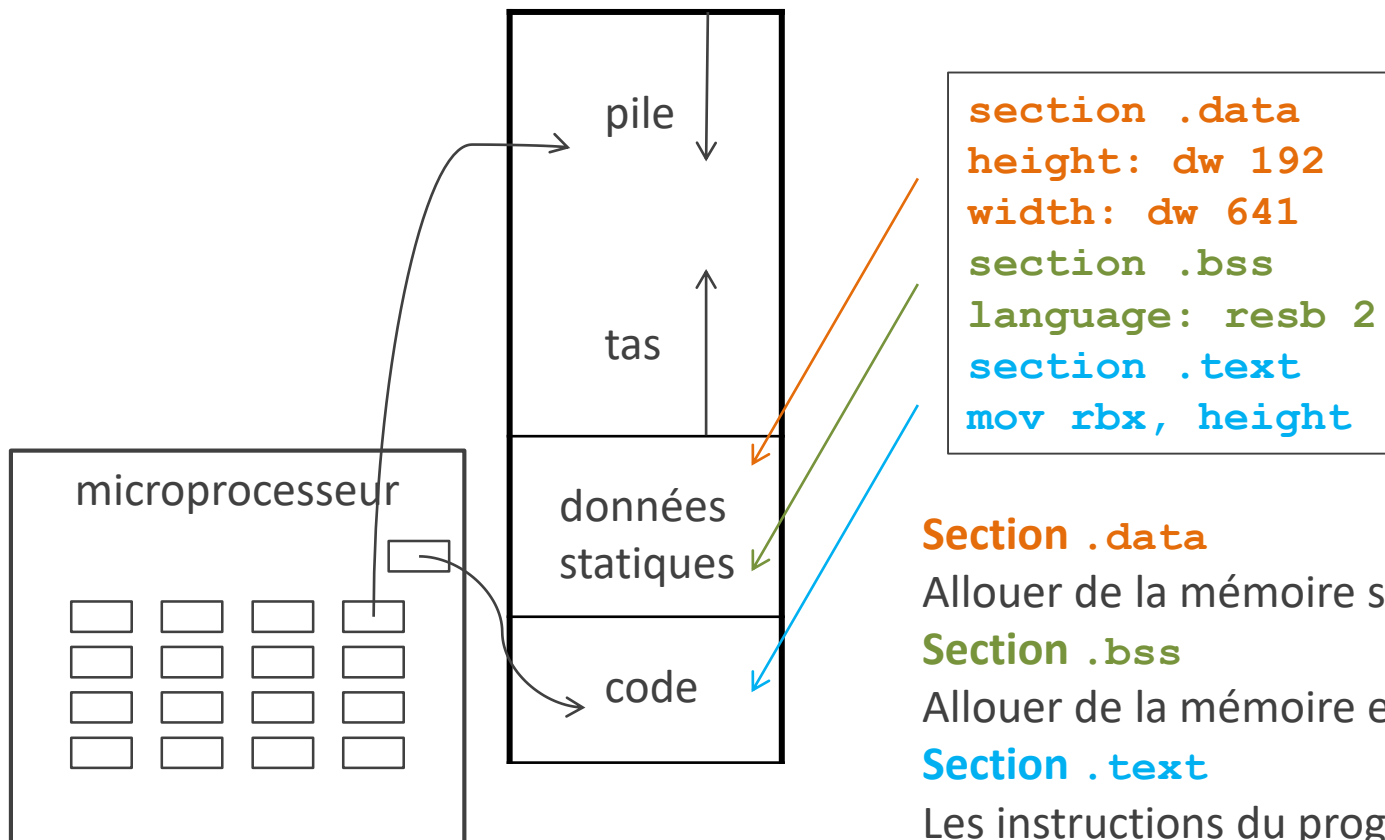
Registres

Boucles

Accès à la mémoire

Terminaison du programme

Allouer de la mémoire statique



Section .data

Allouer de la mémoire statique et initialiser

Section .bss

Allouer de la mémoire et initialiser à 0

Section .text

Les instructions du programme

Section .data Codes

Code		Taille mémoire
db	define byte	1 octet
dw	define word	2 octets (vestige de la version 16 bits)
dd	define double word	4 octets
dq	define quadruple word	8 octets
dt	define ten bytes	10 octets

```
section .data
height: dw 192
width:  dw 641
(...)
section .text
mov rbx, height
```

Forme générale d'une allocation statique

<étiquette> ":" <code> <valeur initiale>

<étiquette> ":" <code> <val1>, <val2>, <val3>...

Le ":" n'est pas obligatoire

Le code fixe la quantité de mémoire réservée (ici 2 octets)

Si plusieurs valeurs initiales, le système alloue à chaque fois la quantité de mémoire indiquée

Section `.data` Étiquette

```
section .data  
height: dw 192  
width:  dw 641  
(...)  
section .text  
mov rbx, height
```

L'étiquette devient une **constante qui vaut l'adresse** de la mémoire réservée (mais ne donne pas la taille réservée)
Toutes les adresses sont codées sur 8 octets
Même taille que `rbx`

Section `.data` Valeurs initiales

mêmes valeurs
en mémoire

```
section .data
height: dw 192
width:  dw 641
numbers: db -2, 254, 0x1
letter:  db "y"
letters: db "y", "n", "?"
singular: db "file"
plural:  dw "files"
folder:  dd "directory"
c_style: db "file", 0
```

Entier

En décimal, hexadécimal ou binaire
Doit tenir dans la mémoire allouée
Peut être un entier signé ou non signé

Caractères

1 ou plusieurs
Entre guillemets (") ou entre apostrophes (')
Si la taille déclarée est trop petite, le système
alloue un multiple de la taille déclarée
Pas de zéro final (au contraire du langage C)
Sauf si on le déclare explicitement

Accès à la mémoire

```
section .data
height: dw 192
width:  dw 641
section .text
mov r12w, word [ height ]
```



Pour accéder à un espace
mémoire il faut 2 données :

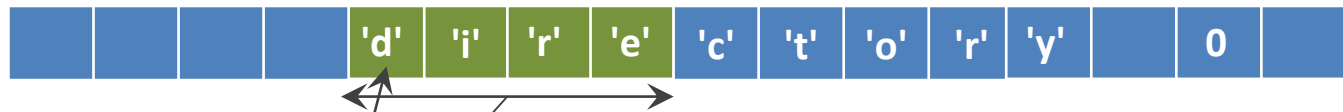
- l'adresse
- la taille

Spécificateur de taille

Attention, ne pas confondre **dw**
(2 octets) et **dword** (4 octets)

Allouer dans .data	Spécificateur de taille	Nombre d'octets
db	byte	1
dw	word	2
dd	dword	4
dq	qword	8
dt	tword	10

Pas de types en `nasm`



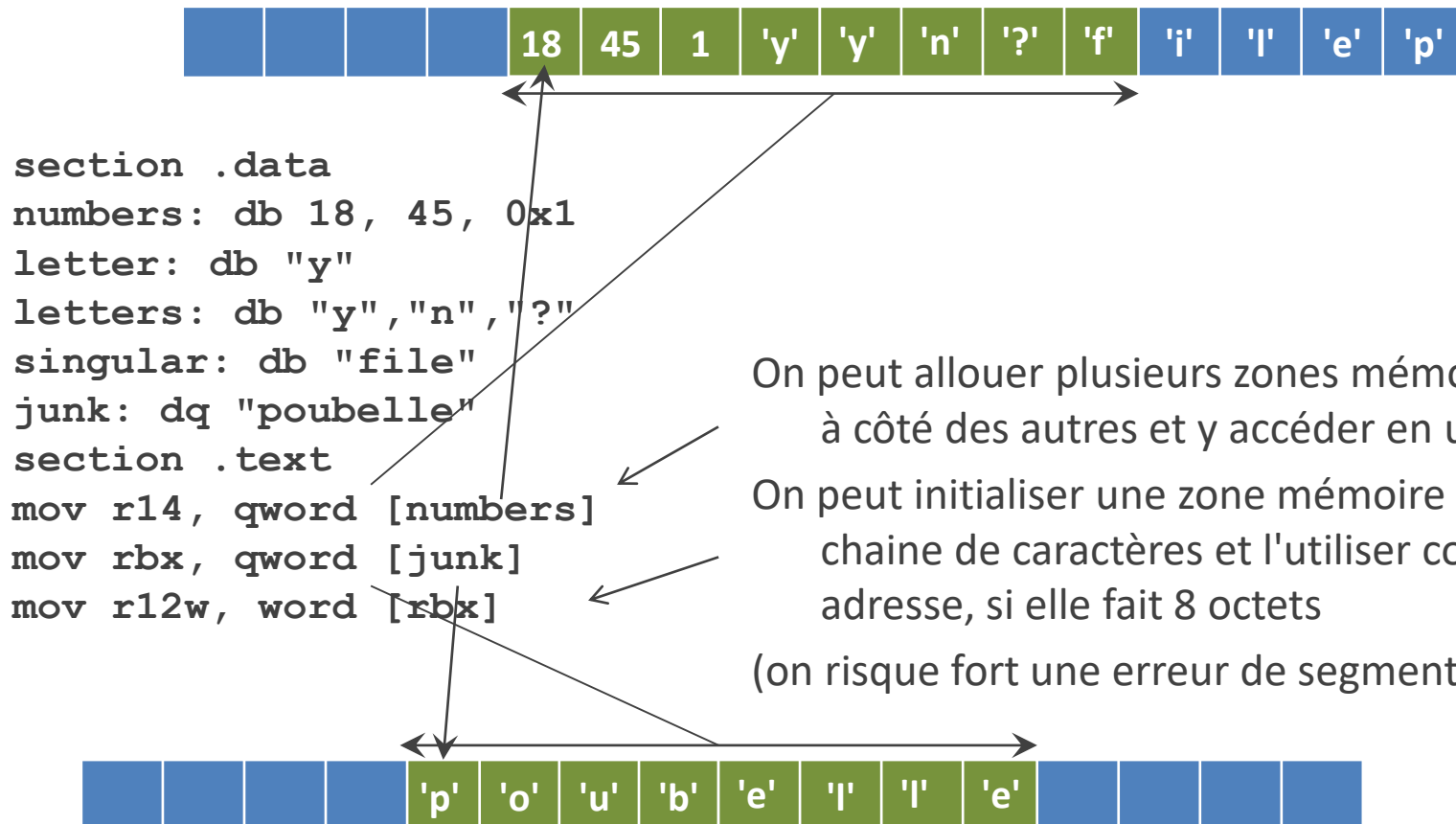
```
section .data
numbers: db 18, 45, 0x1
folder: dt "directory", 0
section .text
mov r13d, dword [folder]
```

Pas de différence entre entier, caractère, chaîne de caractères, adresse mémoire

Tout est champ de bits

On peut allouer une zone mémoire et accéder à une partie de cette zone

Pas de types en `nasm`



On peut allouer plusieurs zones mémoire les unes à côté des autres et y accéder en une fois

On peut initialiser une zone mémoire avec une chaîne de caractères et l'utiliser comme adresse, si elle fait 8 octets

(on risque fort une erreur de segmentation)

Accès mémoire sans spécificateur de taille

```
section .data
height: dw 192
width:  dw 641
section .text
mov r12, 0
mov r12w, [height]
mov r13d, [height]
mov rbx, [height] ; accès téméraire à zone mémoire non allouée
```

Si on accède à la mémoire sans spécifier la taille,
c'est le registre destination qui donne la taille
Cela rend le code moins lisible

```
section .data
height: dw 192
width:  dw 641
section .text
mov r12, 0
mov r12w, word [height]
mov r13w, word [height+2]
mov r14w, word [width]
```

Calcul d'adresses

[registre]

[registre + registre * taille]

taille peut être 1, 2, 4 ou 8

[registre + nombre]

[registre + registre * taille + nombre]

~~[registre - registre * taille]~~

~~[registre + registre * registre]~~

À l'intérieur des crochets on peut mettre certaines expressions avec + ou *

C'est très limité

Le calcul interprète les opérandes comme entiers signés codés en complément à 2

Ce sont les seuls cas où une instruction **nasm** contient une expression arithmétique sans une instruction comme **add** ou **sub**

L'unité pour les adresses est toujours l'octet
contrairement aux indices des tableaux en C

```
int my_numbers[MAX_NUMBERS];
n=my_numbers[i+2];
```

Sommaire

Généralités

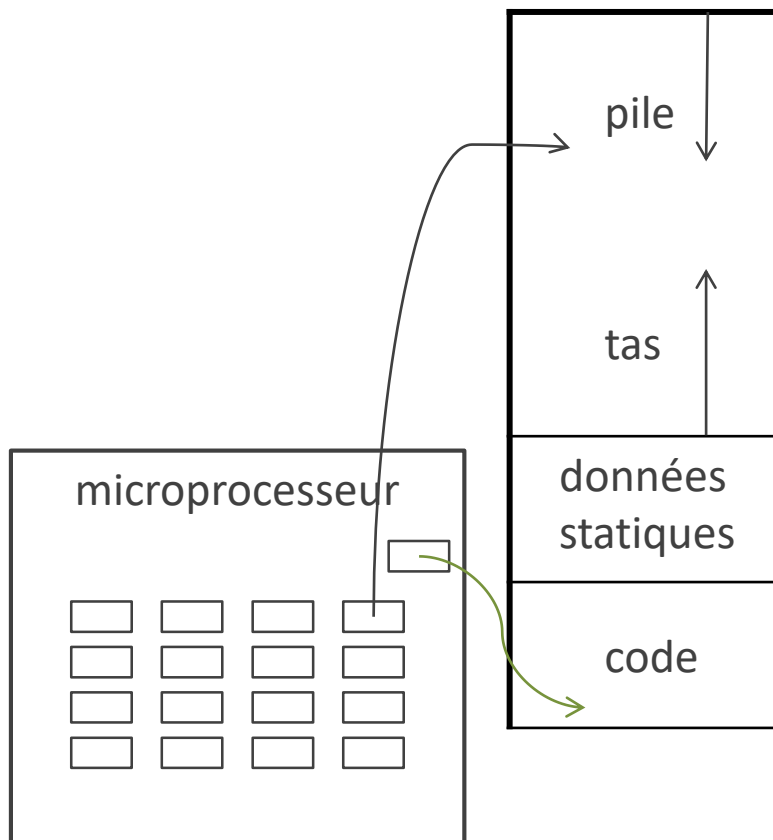
Registres

Boucles

Accès à la mémoire

Terminaison du programme

Terminer le programme



Il faut une instruction de terminaison dans un programme `nasm`

Sinon, le moteur d'exécution passe à l'instruction suivante et peut **sortir de la zone mémoire réservée au code** : erreur à l'exécution

La terminaison du programme est un appel système : libérer la mémoire, terminer le processus...

Instruction `syscall` : la même instruction pour tous les appels système

Appels système avec `syscall`

Appel système	<code>rax</code>	<code>rdi</code>	<code>rsi</code>	<code>rdx</code>
<code>exit</code>	60	valeur de retour		
<code>read</code>	0	fichier	adr. destination	taille
<code>write</code>	1	fichier	adr. source	taille

L'instruction `syscall` a besoin de plusieurs informations

- quel appel système ? `exit`, `read`, `write`...
- si `read` ou `write` : - dans quel fichier ?
 - à quelle adresse ?
 - taille des données ?

```
mov rax, 60
mov rdi, 0
syscall
```

Trop d'informations pour une seule instruction

On les met dans des registres avant d'invoquer `syscall`

La valeur dans `rax` détermine quel appel système est réalisé

Si c'est un `exit`, la valeur dans `rdi` est la valeur de retour

Les principaux registres

8 octets	4 octets	2 octets	1 octet	1 octet
rax	eax	ax	ah	al
rbx	ebx	bx	bh	bl
rcx	ecx	cx	ch	cl
rdx	edx	dx	dh	dl
r12	r12d	r12w		r12b
r13	r13d	r13w		r13b
r14	r14d	r14w		r14b
r15	r15d	r15w		r15b
rsi	esi	si		sil
rdi	edi	di		dil

si : source index

di : destination index

Merci

CONTACT

ERIC LAPORTE

00 +33 (0)1 60 95 75 52

ERIC.LAPORTE@UNIV-EIFFEL.FR